

Candy

Problem Statement: A line of N kids is standing there. The rating values listed in the integer array `ratings` are assigned to each kid. These kids are receiving candy according to the following criteria:

- There must be at least one candy for every child.
- Kids whose scores are higher than their neighbours receive more candies than their neighbours.

Return the minimum number of candies needed to distribute among children.

Examples

Input: `ratings = [1, 0, 5]`

Output: 5

Explanation: The distribution of candies will be 2, 1, 2 to the first, second, and third child respectively.

Input: `ratings = [1, 2, 2]`

Output: 4

Explanation: The distribution of candies will be 1, 2, 1 to the first, second, and third child respectively. The third gets only 1 candy because it satisfies both conditions mentioned above.

Brute force Approach

Algorithm

The objective is to ensure that each child receives at least one candy. Additionally, any child with a higher rating than their adjacent neighbor must receive strictly more candies than that neighbor. The brute force approach simulates this behavior by performing repeated traversals from both directions, adjusting the candy distribution to maintain the constraints.

- Initialize an array to track candies for each child and assign 1 candy to each child (minimum required).
- Iterate repeatedly until no updates are needed in a complete traversal
- Traverse from left to right:
 - For each child, if their rating is greater than the previous child's and their current candy count is not greater, update it to one more than the previous child's candy count.
- Traverse from right to left:
 - For each child, if their rating is greater than the next child's and their current candy count is not greater, update it to one more than the next child's candy count.
- Continue alternating left-to-right and right-to-left traversals until no modifications occur in a full pass.
- Return the total number of candies by summing up the final values in the array.

Dry Run:

ratings = [1, 0, 5]



Candies = { 1, 1, 1 }



left to right pass

for i = 1

1	0	5
---	---	---

rating [0] > rating [1] {no change}

for i = 2

1	0	5
---	---	---

rating [1] < rating [2] candies[2] = candies[1] + 1
and
candies[2] <= candies[1] = 2
Hence update

candies=

1	1	2
---	---	---

 update = true

right to left pass

for i = 1

1	0	5
---	---	---

rating [1] < rating [2] {no update}

for i = 2

1	0	5
---	---	---

rating [0] > rating [1] candies[0] = candies[1] + 1
and
candies[0] <= candies[1] = 2
Hence update

candies=

2	1	2
---	---	---

 update = true

Code

```
import java.util.*;

class Solution {
    // Function to calculate the minimum number of candies
    public int candy(int[] ratings) {
        int n = ratings.length;

        // Array to store candies given to each child, initialized to 1
        int[] candies = new int[n];
        Arrays.fill(candies, 1);

        boolean updated = true;

        // Loop until no changes are made in one full pass
        while (updated) {
            updated = false;

            // Left to right pass
            for (int i = 1; i < n; i++) {
                if (ratings[i] > ratings[i - 1] && candies[i] <= candies[i - 1])
                {
                    candies[i] = candies[i - 1] + 1;
                    updated = true;
                }
            }

            // Right to left pass
            for (int i = n - 2; i >= 0; i--) {
                if (ratings[i] > ratings[i + 1] && candies[i] <= candies[i + 1])
                {
                    candies[i] = candies[i + 1] + 1;
                    updated = true;
                }
            }
        }

        // Sum up the total candies
        int total = 0;
        for (int candy : candies) {
            total += candy;
        }
        return total;
    }
}

// Separate class for main method to run the solution
public class CandyMain {
    public static void main(String[] args) {
        Solution obj = new Solution();
        int[] ratings = {1, 0, 5};
        System.out.println("Minimum candies required: " + obj.candy(ratings));
    }
}
```

Complexity Analysis

Time Complexity: $O(N^2)$ In the worst case, the algorithm can keep updating the candy distribution multiple times. Each full pass (left-to-right and right-to-left) takes $O(N)$ time, where N is the number of children.

Space Complexity: $O(N)$ We use an extra array to store the number of candies given to each child.

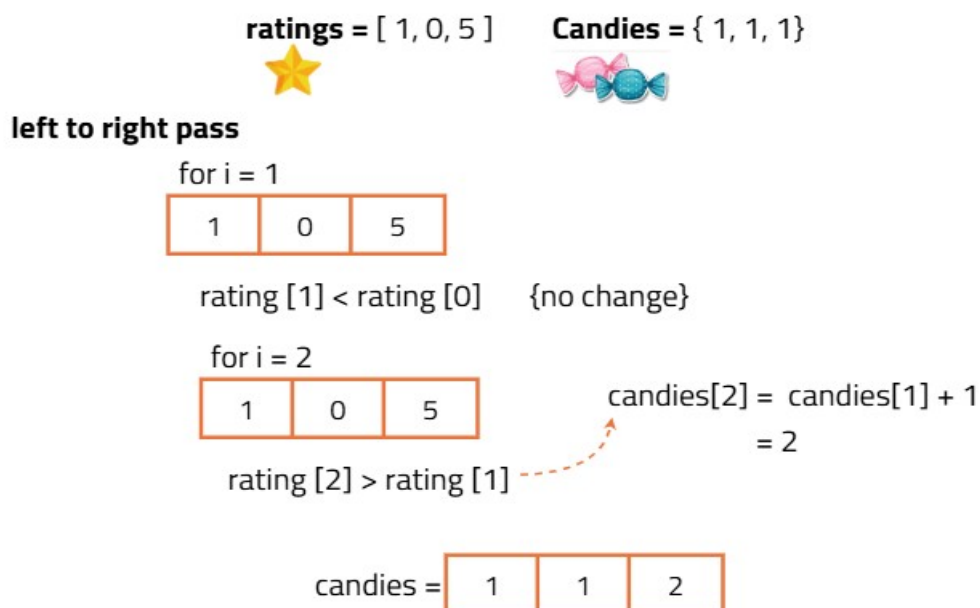
Better Approach

Algorithm

The idea is to distribute candies in two linear passes: one from left to right and one from right to left. In the first pass, we make sure that every child with a higher rating than the left neighbor gets more candies. In the second pass, we do the same but from right to left, ensuring the right neighbor condition is also satisfied. This guarantees that both constraints are met efficiently without unnecessary repeated updates.

- Initialize a candy array where each child gets 1 candy initially.
- Traverse the ratings from left to right:
- If the current child's rating is higher than the previous child, increase the current child's candies to one more than the previous child's candies.
- Traverse the ratings from right to left:
- If the current child's rating is higher than the next child, update the current child's candies to the maximum of current value or one more than the next child's candies.
- After both passes, sum up the values in the candy array to get the minimum total candies required.

Dry Run:



right to left pass

for i = 1

1	0	5
---	---	---

rating [1] < rating [2] {no change}

for i = 2

1	0	5
---	---	---

candies[0] = max (candies[0], candies[1] + 1)
= 2

rating [0] > rating [1]

candies =

2	1	2
---	---	---

total = 5

Code

```
import java.util.*;
class Solution {
    // Function to calculate the minimum number of candies
    public int candy(int[] ratings) {
        int n = ratings.length;
        int[] candies = new int[n];

        // Initialize each child with 1 candy
        for (int i = 0; i < n; i++) {
            candies[i] = 1;
        }

        // Traverse from left to right
        for (int i = 1; i < n; i++) {
            // If current rating is higher than previous, give more candies
            if (ratings[i] > ratings[i - 1]) {
                candies[i] = candies[i - 1] + 1;
            }
        }

        // Traverse from right to left
        for (int i = n - 2; i >= 0; i--) {
            // If current rating is higher than next, adjust candy count
            if (ratings[i] > ratings[i + 1]) {
                candies[i] = Math.max(candies[i], candies[i + 1] + 1);
            }
        }

        // Sum up all candies
        int total = 0;
        for (int c : candies) {
            total += c;
        }
        return total;
    }
}
```

```

}

// Driver class
public class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();
        int[] ratings = {1, 0, 5};
        System.out.println("Minimum candies: " + sol.candy(ratings));
    }
}

```

Complexity Analysis

Time Complexity: $O(n)$, The algorithm makes two passes over the ratings array (left to right and right to left), and a final pass to sum up the candies. Each pass is linear in time.

Space Complexity: $O(n)$, We use an auxiliary array to store the number of candies assigned to each child, which requires $O(n)$ space.

Optimal Approach

Algorithm

Treat the ratings as a combination of increasing and decreasing slopes. At every peak (where the slope changes from increasing to decreasing), we need to give the maximum number of candies. For valleys (local minima), the child should always get 1 candy. The idea is to simulate the shape of the slope: while going up, increase candies; while going down, also increase a counter (valley depth). To avoid double-counting the peak (which is counted from both sides), we subtract the smaller of the two slope heights.

- Start with giving 1 candy to each child, so initialize total candies with size of ratings.
- Iterate through the ratings from left to right using an index.
- If current rating equals the previous one, move ahead (as each already has 1 candy).
- If there's an increasing slope (current rating > previous rating):
 - Count the increasing slope length (peak height).
 - For each step up, add increasing value of candies to total.
- If there's a decreasing slope (current rating < previous rating):
 - Count the decreasing slope length (valley depth).
 - For each step down, add increasing value of candies to total.
- After one increasing and decreasing slope, subtract the smaller of the peak and valley (as the peak child was double-counted).
- Continue this process until the end of the array.
- Return the total candies at the end.

rating[]:

1	3	6	8	9	5	3
---	---	---	---	---	---	---

peak = 0 valley = 0

1	3	6	8	9	5	3
---	---	---	---	---	---	---

$3 > 1$ { increasing slope }

candies = $7 + 1 = 8$

peak = 1 valley = 0

1	3	6	8	9	5	3
---	---	---	---	---	---	---

$6 > 3$ { increasing slope }

candies = $8 + 2 = 10$

peak = 2 valley = 0

1	3	6	8	9	5	3
---	---	---	---	---	---	---

8 > 6 { increasing slope }

candies = 10 + 3 = 13

peak = 3 valley = 0

1	3	6	8	9	5	3
---	---	---	---	---	---	---

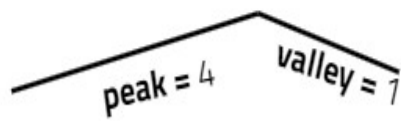
9 > 8 { increasing slope }

candies = 13 + 4 = 17

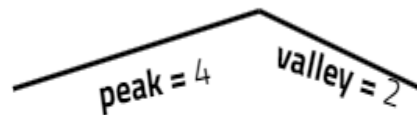
peak = 4 valley = 0



$9 > 5$ { decreasing slope}
candies = $17 + 1 = 18$



$5 > 3$ { decreasing slope}
candies = $18 + 2 = 20$



Now we've reached the end. Subtract $\min(\text{peak} = 4, \text{valley} = 2) = 2$
Total candies = $20 - 2 = 18$

Code

```
class Solution {
    // Function to calculate the minimum number of candies
    public int candy(int[] ratings) {

        // Get number of children
        int n = ratings.length;

        // Initially give 1 candy to each child
        int candies = n;

        // Start from second child
        int i = 1;

        while (i < n) {

            // Skip equal ratings
            if (ratings[i] == ratings[i - 1]) {
                i++;
                continue;
            }

            // Initialize increasing slope counter
            int peak = 0;

            // Traverse strictly increasing ratings
            while (i < n && ratings[i] > ratings[i - 1]) {
                peak++;
                candies += peak;
                i++;
            }

            // Initialize decreasing slope counter
            int valley = 0;

            // Traverse strictly decreasing ratings
            while (i < n && ratings[i] < ratings[i - 1]) {
                valley++;
                candies += valley;
                i++;
            }

            // Remove overlapping candy at the peak
            candies -= Math.min(peak, valley);
        }

        // Return total candies required
        return candies;
    }
}

// Separate main class
public class Main {

    public static void main(String[] args) {

        // Input array of ratings
        int[] ratings = {1, 3, 6, 8, 9, 5, 3};

        // Create object of Solution class
        Solution sol = new Solution();

        // Call function and print result
```

```
        System.out.println("Minimum candies required: " + sol.candy(ratings));  
    }  
}
```

Complexity Analysis

Time Complexity: $O(n)$, We iterate through the ratings array only once using a single while loop with two inner while loops. Each index is processed at most twice: once during an increasing slope and once during a decreasing slope. Hence, the total operations are linear with respect to the size of the input.

Space Complexity: $O(1)$, No extra space is used apart from a few integer variables like peak, valley, i, and candies. No auxiliary arrays or data structures are used, making the space constant regardless of input size.